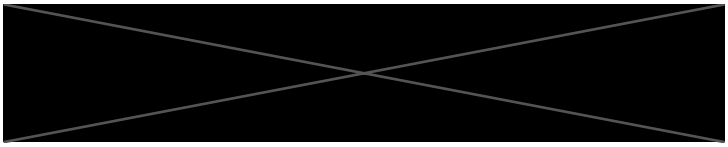
# Final Project: Forecasting Bicycle Theft in the West Midlands (U.K.)

Social Forecasting
Shell Veal-U ham



## Introduction and Data Background

Having previously explored urban cycling patterns using the publicly available Dublin City Bikes dataset (Dublin City Council 2019), I was interested in examining similar seasonal and long-term trends in a different context—namely, the West Midlands in the United Kingdom (UK). The West Midlands is a metropolitan county centred on Birmingham, encompassing the city itself as well as its surrounding suburban areas and neighboring towns.

Directly analyzing cycling ridership in this region is challenging due to the absence of a comprehensive public bicycle sharing scheme, but bicycle theft ought to give us similarly seasonal patterns. Data from data.co.uk (UK Police 2026) provides a record of all such incidents reported to the West Midlands Police from June 2013 until January 2026. Modelling and forecasting this time series presents an opportunity to explore how varied approaches can achieve different results in capturing trends over time in highly seasonally-influenced data.

## Approach

First, I separated the data into a train and test set, before plotting the raw data to visualize its behavior over the years. I then decomposed the time series into its key components—seasonal effects, long-term trend, and residual noise—to better understand the underlying structure. Following this, I fitted a basic Simple Exponential Smoothing (SES) model as an initial forecasting benchmark. I then moved on to fitting a basic ARIMA model to incorporate lagged relationships in the data, before turning to the Seasonal ARIMA (SARMIA) to best capture the seasonality of the data. Finally, I compared all models and visualized their forecasting performance to evaluate how well each approach captured both historical patterns and future behavior.

Before fitting any models, I divided the dataset into training and testing sets. This step is essential for evaluating how well a model generalizes to unseen data—that is, how accurately

it can predict future observations rather than simply fitting past ones. In this case, I used 80% of the data (spanning May 2013 to June 2023) as the training set, and set aside the remaining 20% (July 2023 to January 2026) as the test set ("future" data). All models were fitted exclusively on the training data and subsequently evaluated on the test data.

Initial visualizations of the decomposed time series revealed two key patterns: a strong seasonal signal and a gradual downward trend over time. The presence of seasonality makes sense, as bicycle theft is closely linked to cycling activity, which itself varies with weather conditions. Warmer, drier months tend to see increased bicycle use, while colder, wetter periods typically correspond to reduced ridership and, it would seem, fewer thefts. The observed downward trend could reflect several underlying factors. Improvements in regional anti-theft measures, the development of alternative transport infrastructure, or a broader decline in cycling as a primary mode of transport might all contribute to such a pattern.

The downward trend over time is also indicative of a changing mean, suggesting we will likely need our models to difference the data once or twice. I kept this in mind throughout my process of model development.

# Model Development and Comparison

I began with the Simple Exponential Smoothing (SES) approach. This method forecasts future values by taking a weighted average of past observations, giving more importance to recent data. It works well as a baseline because it captures the overall level of the series without making strong assumptions about trends or patterns.

The SES model (explicitly shown in equation 1 below) didn't seem to fully account for the impact of past values on current ones, so I moved on to testing explicitly defined lags with my base ARIMA model. Beginning with once-lagged data and concluding that twice-lagged data made the best improvements, my ARIMA model "learns" from previous twice-lagged time steps rather than just smoothing them with the weighted average (as SES does). I also added a once-lagged moving average component to control for the lagged impact of shocks in the data (making my final non-seasonal model of the following structure: ARIMA (2, 0, 1) - see equation 2 for specifics).

Given my initial observation of a downward trend over time, I had experimented with differentiation in the ARIMA model (once or twice differencing to the data can stabilize the mean over time, better accommodating the "stationarity" assumption of the ARIMA model). Adding a differencing component to the base ARIMA model, however, vastly increased the error metrics, suggesting that the differencing might best be conducted at the seasonal level.

Theoretical understandings of bicycle theft suggest a seasonal pattern, and my initial visualizations confirm this. To capture the seasonality, I extended my project to developing a Seasonal ARIMA (SARIMA), which incorporates repeating seasonal effects alongside the lagged structure. Adjusting for optimal error metrics without delving into extreme complexity, I landed on an SARIMA model that had one lag on the non-seasonal level, one lag on the seasonal level, once-differentiation at the seasonal level, and a once-lagged moving average component to compensate for the autocorrelation of residuals in the data (ARIMA (1,0,1)(1,1,0) - see equation 3 for specifics).

Looking at the results in Table 1, we can see a clear progression in model performance (Mean Average Error (MAE) and Mean Average Percent Error (MAPE) decreasing) as complexity increases. Interestingly, the SES model slightly outperforms the non-seasonal ARIMA model on the test data. This suggests that the ARIMA model may be overfitting—capturing noise in the training data that doesn't generalize well to new data. SES, being simpler, can sometimes avoid this pitfall. Overall, the Seasonal ARIMA model performs best, indicating that incorporating both lagged relationships and seasonal patterns provides the most accurate and reliable forecasts for bicycle theft in the West Midlands.

Table 1: Model Error Comparison (Train vs Test)

| | **Training** | | **Test** | |
|---|---|---|---|---|
| Model | Train_MAE | Train_MAPE | Test_MAE | Test_MAPE |
| SES | 45.17 | 22.56 | 43.93 | 32.85 |
| ARIMA (Plain) | 37.16 | 18.14 | 47.02 | 35.47 |
| ARIMA (Seasonal) | 30.71 | 14.41 | 40.73 | 25.11 |

## Equations

**1: Simple Exponential Smoothing (SES) with $\alpha = 0.05$ :**
Level equation:

$$\ell_t = 0.5y_t + 0.5\ell_{t-1}$$

Forecast (constant past final "t" observed):

$$\hat{y}_{t+h} = \ell_t$$

**2: ARIMA(2,0,1) (non-seasonal):**

$$y_t = 1.68y_{t-1} - 0.68y_{t-2} - 0.96\varepsilon_{t-1} + \varepsilon_t$$

**3: Seasonal ARIMA(1,0,1)(1,1,0)[12]:**
Let $B$ be the backshift operator. The model is:

$$(1 - B^{12})y_t = 0.74(1 - B^{12})y_{t-1} - 0.26\varepsilon_{t-1} - 0.42(1 - B^{12})y_{t-12} + \varepsilon_t$$

**Notation:**

- $t$ – indexes time (in this case, monthly observations).
- $y_t$ – bicycle theft at time $t$.
- $\hat{y}_t$ – predicted bicycle theft at time $t$.
- $\varepsilon_t$ – white noise error term at time $t$, assumed to have mean 0.
- $[(1 - B^{12})y_t = y_t - y_{t-12}]$ – represents (one full year of) seasonal differencing.
- $\ell_t$ – the level component in exponential smoothing at time $t$.

### Visualizing Model Comparisons

As shown in Figure 1, the Simple Exponential Smoothing (SES) model produces relatively wide confidence intervals, which expand significantly over the forecast horizon. In some cases, the lower bound of these intervals even dips into negative values, which is not meaningful in the context of bicycle theft.

In contrast, the basic ARIMA model produces noticeably tighter prediction intervals. The inclusion of lagged relationships helps stabilise the forecasts over time, meaning the uncertainty does not increase as rapidly as it does under SES. This results in more consistent behavior across the forecasting horizon.

However, the strongest performance is clearly the SARIMA model, the forecasting power of which is much more attuned to the data. By explicitly accounting for seasonal variation in differencing *and* lag, this model is able to track the data much more closely in the test set (i.e., the "future" period). It not only provides improved point forecasts but also offers more realistic uncertainty bounds, reflecting the recurring structure present in bicycle theft patterns.

## Conclusions

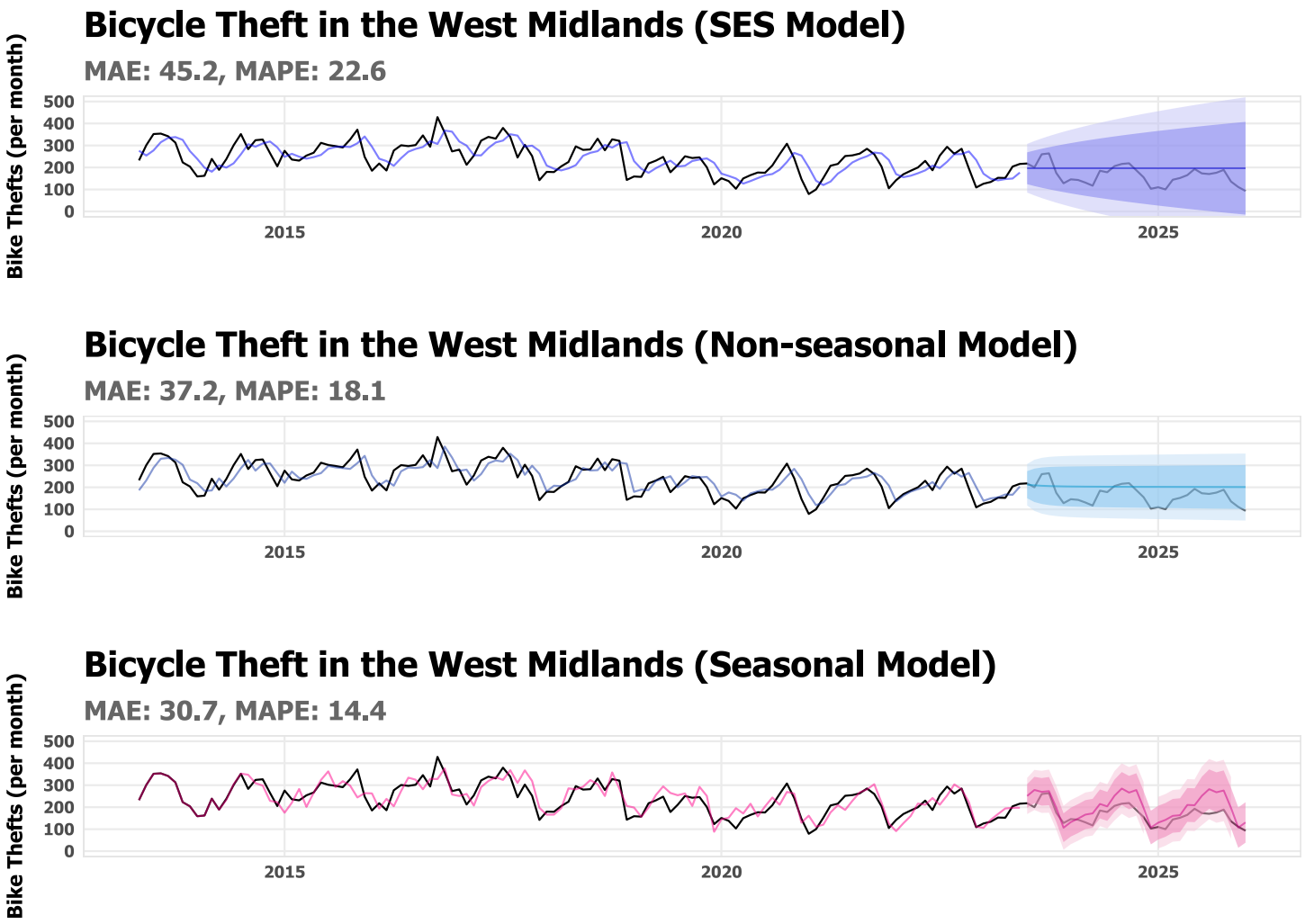
The three models explored in this report can support different goals relating to data modeling and forecasting.

If the priority is to ensure that all possible variation in bicycle theft is captured, even at the expense of precision, then Simple Exponential Smoothing (SES) is the most conservative choice. Its comparatively wide confidence intervals mean that it is more likely to contain the true values, but this comes with a lack of specificity and weaker structure in terms of capturing underlying dynamics.

If, instead, the goal is to produce a reasonable average forecast that performs well most of the time, then a basic ARIMA model is an improvement. It incorporates lagged relationships in the data and produces more stable estimates than SES. However, given that the inclusion of once-differencing at the non-seasonal level introduced a lot of error and I decided not to differentiate, my non-seasonal ARIMA model has limitations in capturing the long-term downward trend in full. In particular, its forward projections tend to flatten into a near-linear path, which may understate gradual change in bicycle theft over time.

Ultimately, the strongest model for both forecasting accuracy and structural realism is the Seasonal ARIMA (SARIMA). By explicitly incorporating seasonal patterns alongside trend and lagged effects, it provides the most faithful representation of the data's behavior. It consistently produces tighter, more realistic forecasts and best reflects both the cyclical nature of bicycle theft and its longer-term evolution. Still, there are moments in the data where we can observe the actual "future" data dipping outside of the first (95%) confidence interval into the 80% interval, whereas for the SES model, all observed data remain within the 95% confidence interval.

Figure 1: SES, ARIMA, SARIMA Models




# Appendix 1: Citations


Dublin City Council. (2019). Dublinbikes API DCC [Data set]. Data.gov.ie. https://data.gov.ie/dataset/dublinbikes-api

OpenAI. (2026). ChatGPT (utilized for debugging R code, rephrasing report, and equation and bibliography formatting [Accessed 2026-04-24].

UK Police. (2026). Street-level crime data: West Midlands (2013–present) [Custom data set]. https://data.police.uk/data/

# Appendix 2: Project Code

```r
# Reading in / Preprocessing the Data
birmingham_data <- read_csv("birmingham_2010_2026.csv")

bdf <- birmingham_data %>%
  select(c(`Crime type`, "Month")) %>%
  filter(`Crime type` == "Bicycle theft") %>%
  mutate(Month = ymd(paste0(Month, "-01"))) %>%
  group_by(Month) %>%
  summarize(observation_count = n())
# Creating a Time Series Object
bts <- ts(bdf$observation_count, start = c(2013, 5),
          end = c(2026, 1),
          frequency = 12)
# Splitting into Train and Test Sets
bts_train <- bts %>% window(start = c(2013, 5),
                            end = c(2023, 6),
                            frequency = 12)
bts_test <- bts %>% window(start = c(2023, 7),
                           end = c(2026, 1),
                           frequency = 12)
##############
# Analysis
##############

# Initial look at the data:
autoplot(bts_train) + ylim(0, 450) +
  autolayer(bts_test, color = "#FF008A", alpha = 0.5)

# Once Differentiated:
once_diff <- bts_train %>% diff(lag = 12)
autoplot(once_diff) +
  geom_hline(yintercept = 0, color = "red", alpha = 0.5)

# Twice Differentiated:
twice_diff <- bts_train %>% diff(lag = 12) %>% diff(lag = 1)
autoplot(twice_diff) +
  geom_hline(yintercept = 0, color = "red", alpha = 0.5)

# Decomposing the data to have a look at components:
stl_b <- stl(bts_train, s.window = 12)
stl_once_diff <- stl(once_diff, s.window = 12)
stl_twice_diff <- stl(twice_diff, s.window = 12)

a <- autoplot(stl_b) +
  labs(title = "Basic Data") +
  geom_hline(yintercept = 0, color = "red", alpha = 0.5)
b <- autoplot(stl_once_diff) +
  labs(title = "Once Differentiated Data") +
  geom_hline(yintercept = 0, color = "red", alpha = 0.5)
```

```r
c <- autoplot(stl_twice_diff) +
  labs(title = "Twice Differentiated Data") +
  geom_hline(yintercept = 0, color = "red", alpha = 0.5)

# Plain data vs Once-Differentiated
a+b

# Once-Differentiated vs Twice-Differentiated
b+c

# Fitting an SES model
b_ses <- ses(bts_train, alpha = 0.5, h = 31)

# Visualizing
fit_ses <- autoplot(bts_train) +
  autolayer(fitted(b_ses), color = "blue", alpha = 0.5) +
  theme(legend.position = "none")

predict_ses <- autoplot(bts) +
  labs(title = "Bicycle Theft in the West Midlands (SES Model)",
       subtitle = "MAE: 45.2, MAPE: 22.6",
       x = "",
       y = "Bike Thefts (per month)") +
  coord_cartesian(ylim = c(0, 500)) +
  autolayer(fitted(b_ses), color = "blue", alpha = 0.5) +
  autolayer(b_ses, color = "blue", alpha = 0.5) +
  theme(legend.position = "none")

train_MAE_SES <- 45.1706
train_MAPE_SES <- 22.56231
test_MAE_SES <- mean(abs(bts_test - tail(b_ses$mean, n = 31)))
test_MAPE_SES <- mean(abs((bts_test - tail(b_ses$mean, n = 31))/bts_test))
    *100

# Checking specs, visualizing error, autocorrelation:
summary(b_ses)
checkresiduals(b_ses)

# Trying Arima with lags 1 and 2
lag_1 <- arima(bts_train, order = c(1, 0, 0))
lag_2 <- arima(bts_train, order = c(2, 0, 0))

forecast(lag_1, h = 31)

plot_lag_1 <- autoplot(bts) +
  autolayer(fitted(lag_1), color = "seagreen", alpha = 0.5) +
  theme(legend.position = "none") +
  coord_cartesian(ylim = c(0, 500)) +
  autolayer(forecast(lag_1, h = 31), color = "darkred", alpha = 0.5)
plot_lag_2 <- autoplot(bts) +
  autolayer(fitted(lag_2), color = "seagreen", alpha = 0.5) +
```

```
  theme(legend.position = "none") +
  coord_cartesian(ylim = c(0, 500)) +
  autolayer(forecast(lag_2, h = 31), color = "darkred", alpha = 0.5)

plot_lag_1 + predict_ses
plot_lag_2 + predict_ses

# Checking the Residuals of ARIMA(1,0,0)
checkresiduals(lag_1)
# Checking the Resiudals of ARIMA(2,0,0)
checkresiduals(lag_2)

# lag 2 better, there's definitely a seasonal issue.

# Fitting and evaluating ARIMA
manual_plain <- arima(bts_train, order = c(2, 0, 1),
                      include.mean = FALSE)
summary(manual_plain)
ACF_manual <- ggAcf(manual_plain$residuals)
PACF_manual <- ggPacf(manual_plain$residuals)
ACF_manual + PACF_manual

# Extracting ARIMA MAE for train and test data
train_MAE_plain <- 37.15893
train_MAPE_plain <- 18.14114
man_plain_fc <- forecast(manual_plain, h = 31)
test_MAE_plain <- mean(abs(bts_test - man_plain_fc$mean))
test_MAPE_plain <- mean(abs((bts_test - man_plain_fc$mean)/bts_test))*100

# Fitting and evaluating SARIMA
manual_seasonal <- arima(bts_train, order = c(1, 0, 1),
                         seasonal = list(order = c(1, 1, 0), period = 12),
                         include.mean = FALSE)
summary(manual_seasonal)
ACF_manual_s <- ggAcf(manual_seasonal$residuals)
PACF_manual_s <- ggPacf(manual_seasonal$residuals)
ACF_manual_s + PACF_manual_s

# Extracting SARIMA MAE for train and test data
train_MAE_seasonal <- 30.71295
train_MAPE_seasonal <- 14.41497
man_seas_fc <- forecast(manual_seasonal, h = 31)
test_MAE_seasonal <- mean(abs(bts_test - man_seas_fc$mean))
test_MAPE_seasonal <- mean(abs((bts_test - man_seas_fc$mean)/bts_test))*
    100

# Final Visualizations:
MANUAL_PLAIN_PLOT <- autoplot(bts) +
  autolayer(fitted(manual_plain),
            color = "#1134A6", alpha = 0.5) +
  autolayer(forecast(manual_plain, h = 31),
```

```
            color = "#0080FE", alpha = 0.5) +
  labs(title = "Bicycle Theft in the West Midlands (Non-seasonal Model)",
       subtitle = "MAE: 37.2, MAPE: 18.1",
       x = "",
       y = "Bike Thefts (per month)") +
  ylim(0,500)

MANUAL_PLOT_SEASONAL <- autoplot(bts) +
  autolayer(fitted(manual_seasonal),
            color = "#FF008A", alpha = 0.5) +
  autolayer(forecast(manual_seasonal, h = 31),
            color = "#FF408A", alpha = 0.5) +
  labs(title = "Bicycle Theft in the West Midlands (Seasonal Model)",
       subtitle = "MAE: 30.7, MAPE: 14.4",
       x = "",
       y = "Bike Thefts (per month)") +
  ylim(0,500)

figure <- predict_ses /MANUAL_PLAIN_PLOT / MANUAL_PLOT_SEASONAL

# ggsave("figure.pdf", figure, width = 11,
#        height = 8,
#        units = "in",
#        dpi = 300)

# Comparing results with auto.arima
auto <- auto.arima(bts_train)

ACF_auto <- ggAcf(auto$residuals)
PACF_auto <- ggPacf(auto$residuals)

ACF_auto + PACF_auto
accuracy(auto)

AUTO_PLOT <- autoplot(bts) +
  autolayer(fitted(auto), color = "#1134A6", alpha = 0.5) +
  autolayer(forecast(auto, h = 31), color = "#0080FE", alpha = 0.5) +
  labs(title = "Bicycle Theft in the West Midlands",
       y = "Bike Theft (per month)")
```